

ACTIVIDAD

Dossier de Actividades Obligatorias del Tema: 4.
Caracterización de modelos computacionales de redes neuronales (Deep Learning)

Máster IA y Big Data

MP2: Sistemas de aprendizaje automático

ÍNDICE

EJERCICIO 1	3
Punto (2, 3) Clase 1	3
Punto (4, 5) Clase 1	5
Punto (1,1) Clase 0	5
Punto (0,2) Clase 0	5
Predicción final para el punto (1,2)	5
EJERCICIO 2	6
Paso 1: Esquina superior izquierda	6
Paso 2: Esquina superior derecha (desplazamiento a la derecha)	7
Paso 3: Esquina inferior izquierda (desplazamiento hacia abajo)	7
Paso 4: Esquina inferior derecha (desplazamiento a la derecha)	7
Matriz Resultante Final	7
EJERCICIO 3	8
La estructura de la red	8
Describe qué valores hemos elegido para los siguientes parámetros en la implementación	9

EJERCICIO 1 [-Ir al Índice-](#)

Un perceptrón es el modelo más simple de una red neuronal artificial. Consta de una única neurona que toma varias entradas, las combina linealmente con pesos asignados, y pasa el resultado a través de una función de activación para producir una salida.

Tenemos un conjunto formado por cuatro puntos de 2 dimensiones: (2,3) y (4,5) pertenecen a la clase 1, (1,1) y (0,2) pertenecen a la clase 0. A partir de estos datos de entrada, de los siguientes pesos iniciales $w_1 = 0,5$, $w_2 = -0,5$ y $b=0$, de una tasa de aprendizaje $\eta=0,1$ y de la función de activación ReLU, ejecuta paso por paso una iteración del proceso de entrenamiento del perceptrón con los cuatro inputs.

Al finalizar, obtén la predicción para el punto (1,2).

Como he venido haciendo hasta ahora, voy a ir explicando paso a paso, justamente para que yo pueda entender lo que estoy haciendo y para que mi yo del futuro pueda entenderlo igualmente.

Según visto hasta ahora, si no me equivoco, una neurona hace estos pasos:

1. Recibe entradas (x_1, x_2).
2. Multiplica cada entrada por su peso (w_1, w_2).
3. Suma los resultados y añade el sesgo (b).
4. Obtiene un valor intermedio z .
5. Aplica la función de activación (ReLU) sobre z .

La fórmula que resulta de lo anterior es:

$$z = x_1 * w_1 + x_2 * w_2 + b$$

Y la función de activación (función matemática que toma el valor z obtenido en el punto anterior y la transforma en una salida) ReLU es:

$$\text{ReLU}(z) = \max(0, z)$$

Los pesos iniciales indicados en el enunciado son los siguientes:

$$\begin{aligned}w_1 &= 0.5 \\w_2 &= -0.5 \\b &= 0 \\\eta &= 0.1\end{aligned}$$

Y según los datos del ejercicio, tenemos que los cuatro puntos (con sus respectivas clases) del conjunto son:

Punto	Clase
(2,3)	1
(4,5)	1
(1,1)	0
(0,2)	0

Punto (2, 3) | Clase 1 [-Ir al Índice-](#)

Multiplicamos cada entrada por su peso correspondiente y sumamos el sesgo:

$$\begin{aligned}z &= 2 \times 0.5 + 3 \times (-0.5) + 0 \\z &= 1 + (-1.5) + 0 \\z &= -0.5\end{aligned}$$

Si lo dejamos en una única línea, tenemos que:

$$z = 2 \times 0.5 + 3 \times (-0.5) + 0 = 1 + (-1.5) + 0 = -0.5$$

Aplicamos la función de activación, pasamos el resultado por "la **ReLU**":

$$\begin{aligned}\text{ReLU}(z) &= \max(0, z) \\ \text{ReLU}(-0.5) &= \max(0, -0.5) \\ \text{ReLU}(-0.5) &= 0\end{aligned}$$

$$\hat{y} = \max(0, z) = \max(0, -0.5) = 0$$

Por tanto, la **predicción actual** ($\hat{y}$) del **perceptrón** para el punto **(2,3)** es:

$$\hat{y} = 0$$

Y el cálculo del **Error** sería:

$$\text{Error} = y - \hat{y} = 1 - 0 = 1$$

Hasta aquí, entiendo que acabamos de completar los dos primeros pasos del entrenamiento:

- **Forward propagation**, calcular la salida de la neurona: $\hat{y} = 0$
- **Cálculo del error**, comparar la predicción con la clase real: **Error = 1**

Entiendo que **como el planteamiento del ejercicio no menciona nada** de función de pérdida, fórmula del gradiente, derivada a utilizar, o algoritmo de Backpropagation y sólo detalla los 4 puntos, pesos y sesgo iniciales, tasa de aprendizaje, de la función ReLU y de ejecutar una iteración, **vamos a usar el método clásico del perceptrón** (espero no equivocarme).

Para los pesos $\omega_{(\text{nuevo})} = \omega_{(\text{actual})} + \eta * \text{Error} * x$

Para el sesgo $b_{(\text{nuevo})} = b_{(\text{actual})} + \eta * \text{Error}$

Para el primer peso ω_1 :

$$\begin{aligned}\omega_{1(\text{nuevo})} &= 0.5 + 0.1 \times 1 \times 2 \\ \omega_{1(\text{nuevo})} &= 0.5 + 0.2 \\ \omega_{1(\text{nuevo})} &= 0.7\end{aligned}$$

$$\omega_{1(\text{nuevo})} = 0.5 + 0.1 \times 1 \times 2 = 0.5 + 0.2 = 0.7$$

Para el segundo peso ω_2 :

$$\omega_{2(\text{nuevo})} = (-0.5) + 0.1 \times 1 \times 3 = (-0.5) + 0.3 = -0.2$$

Para el sesgo (b)

$$b_{(\text{nuevo})} = 0 + 0.1 \times 1 = 0 + 0.1 = 0.1$$

Por tanto, tras entrenar con el primer patrón:

$$\begin{aligned}w_1 &= 0.7 \\ w_2 &= -0.2 \\ b &= 0.1\end{aligned}$$

Punto (4, 5) | Clase 1 [-Ir al Índice-](#)

Multiplicamos cada entrada por su peso correspondiente y sumamos el sesgo:

$$z = 4 \times 0.7 + 5 \times (-0.2) + 0.1 = 2.8 + (-1) + 0.1 = 1.9$$

$$\hat{y} = \max(0, z) = \max(0, 1.9) = 1.9$$

$$\text{Error} = y - \hat{y} = 1 - 1.9 = -0.9$$

Actualizamos los pesos (ω_1 , ω_2) y el sesgo (b)

$$\omega_{1(\text{nuevo})} = 0.7 + 0.1 \times (-0.9) \times 4 = 0.7 + (-0.36) = 0.34$$

$$\omega_{2(\text{nuevo})} = (-0.2) + 0.1 \times (-0.9) \times 5 = -0.2 + (-0.45) = -0.65$$

$$b_{(\text{nuevo})} = 0.1 + 0.1 \times (-0.9) = 0.1 + (-0.09) = 0.01$$

Tras entrenar:

$$w_1 = 0.34$$

$$w_2 = -0.65$$

$$b = 0.01$$

Punto (1,1) | Clase 0 [-Ir al Índice-](#)

$$z = 1 \times 0.34 + 1 \times (-0.65) + 0.01 = 0.34 + (-0.65) + 0.01 = -0.3$$

$$\hat{y} = \max(0, z) = \max(0, -0.3) = 0$$

$$\text{Error} = y - \hat{y} = 0 - 0 = 0$$

$$\omega_{1(\text{nuevo})} = 0.34 + 0.1 \times 0 \times 1 = 0.34 + 0 = 0.34$$

$$\omega_{2(\text{nuevo})} = (-0.65) + 0.1 \times 0 \times 1 = -0.65 + 0 = -0.65$$

$$b_{(\text{nuevo})} = 0.1 + 0.1 \times 0 = 0.1 + 0 = 0.01$$

Podemos deducir que como el Error es 0, tras entrenar los parámetros no se modifican:

$$w_1 = 0.34$$

$$w_2 = -0.65$$

$$b = 0.01$$

Punto (0,2) | Clase 0 [-Ir al Índice-](#)

$$z = 0 \times 0.34 + 2 \times (-0.65) + 0.01 = 0 + (-1.3) + 0.01 = -1.29$$

$$\hat{y} = \max(0, z) = \max(0, -1.29) = 0$$

$$\text{Error} = y - \hat{y} = 0 - 0 = 0$$

Nuevamente, el error es 0, por lo que los parámetros se quedan exactamente igual para el final de la iteración:

$$w_1 = 0.34$$

$$w_2 = -0.65$$

$$b = 0.01$$

Predicción final para el punto (1,2) [-Ir al Índice-](#)

$$z = 1 \times 0.34 + 2 \times (-0.65) + 0.01 = 0.34 + (-1.3) + 0.01 = -0.95$$

$$\hat{y} = \max(0, z) = \max(0, -0.95) = 0$$

La **predicción** final del perceptrón **para** el punto (1,2) es 0

EJERCICIO 2 [-Ir al Índice-](#)

Vamos a realizar paso por paso una iteración el proceso de convolución análogo al que realizan las CNN. Por ejemplo, las imágenes en blanco y negro se representan como una matriz de píxeles, donde la dimensión de la matriz es la resolución de la imagen (es decir, una imagen 256x256 es una matriz de ese mismo tamaño) donde cada valor corresponde al color del píxel en una escala de grises.

Vamos a hacer un ejemplo partir de la siguiente matriz :

1	2	3
4	5	6
7	8	9

Sobre esta matriz vamos a aplicar el siguiente filtro de convolución 2x2 (este filtro se corresponde con los pesos de las capas de convolución, que se van optimizando durante el entrenamiento):

1	0
-1	1

Realiza una iteración del proceso de convolución, ejecutando los siguientes pasos:

- Posiciona el filtro en la esquina superior izquierda de la imagen.
- Multiplica cada elemento del filtro por el elemento correspondiente de la imagen.
- Suma los resultados de estas multiplicaciones para obtener un solo valor.
- Desplaza el filtro una posición a la derecha y repite los pasos 2 y 3.
- Continúa desplazando el filtro a la derecha y luego hacia abajo, repitiendo los pasos hasta cubrir toda la imagen.

Ejecuta una iteración de este proceso, el resultado debe ser una matriz 2x2.

Como siempre, voy a intentar operar de forma organizada para entender el paso a paso del desarrollo del ejercicio.

Paso 1: Esquina superior izquierda [-Ir al Índice-](#)

Lo primero, posicionar el filtro en la esquina superior izquierda de la imagen, así que, posicionamos (literalmente) el filtro sobre los píxeles superiores izquierdos de la imagen

1 ₁	0 ₂	3
-4 ₄	1 ₅	6
7	8	9

Región de la imagen seleccionada:

1	2
4	5

Vamos a “multiplicar cada elemento del filtro por el elemento correspondiente de la imagen”

$$z_{1,1} = (1 \times 1) + (0 \times 2) + (-1 \times 4) + (1 \times 5)$$

$$z_{1,1} = 1 + 0 + (-4) + 5 = 2$$

Paso 2: Esquina superior derecha (desplazamiento a la derecha) [-Ir al Índice-](#)

1	2	3
4	5 ¹	6
7	8	9



Región de la imagen seleccionada:

2	3
5	6

$$z_{1,2} = (1 \times 2) + (0 \times 3) + (-1 \times 5) + (1 \times 6)$$

$$z_{1,2} = 2 + 0 + (-5) + 6 = 3$$

Paso 3: Esquina inferior izquierda (desplazamiento hacia abajo) [-Ir al Índice-](#)

1	2	3
4 ¹	5 ⁰	6
7 ⁻¹	8 ¹	9

Región de la imagen seleccionada:

4	5
7	8

$$z_{2,1} = (1 \times 4) + (0 \times 5) + (-1 \times 7) + (1 \times 8)$$

$$z_{2,1} = 4 + 0 + (-7) + 8 = 5$$

Paso 4: Esquina inferior derecha (desplazamiento a la derecha) [-Ir al Índice-](#)

1	2	3
4	5 ¹	6 ⁰
7	8 ⁻¹	9 ¹

Región de la imagen seleccionada:

5	6
8	9

$$z_{2,1} = (1 \times 5) + (0 \times 6) + (-1 \times 8) + (1 \times 9)$$

$$z_{2,1} = 5 + 0 + (-8) + 9 = 6$$

Matriz Resultante Final [-Ir al Índice-](#)

Agrupando los cuatro valores obtenidos en las respectivas posiciones, la matriz resultante es:

2	3
5	6

EJERCICIO 3 [-Ir al índice-](#)

Keras es uno de los frameworks más conocidos para trabajar con redes neuronales. Revisa la documentación oficial de los siguientes elementos de Keras (`Sequential` de `keras.models` y `Dense` de `keras.layers`) si es necesario, para entender la siguiente implementación de una FNN en Keras.

```
model.add(Dense(units=64, activation='relu', input_dim=10))
model.add(Dense(units=64, activation='relu'))
model.add(Dense(units=5, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Describe qué valores hemos elegido para los siguientes parámetros en la implementación:

- Número de capas
- Volumen de cada capa
- Función de activación de cada capa
- Optimizador
- Función de pérdida
- Métrica de evaluación

Por último, con esta red neuronal que estamos implementando, ¿Qué tipo de problema estamos intentando resolver?

Siguiendo la misma dinámica, me voy a explicar la estructura para tener esto de repaso o apuntes y, posteriormente, intentar “describir —más rápidamente— qué valores se han elegido para los siguientes parámetros en la implementación”.

La estructura de la red [-Ir al índice-](#)

Capa 1

```
model.add(Dense(units=64, activation='relu', input_dim=10))
```

Construye una capa, la primera del modelo, como si de una lista se tratase a la que le hacemos un `.append()`. Al ser la primera de las capas que estamos creando, Keras obliga a indicar cuántos datos van a entrar por primera vez a la red. Esta primera capa tiene doble función porque es la que recibe el impacto de datos de entrada, pero como sus neuronas procesan la información funciona también como oculta del sistema.

Con el parámetro `units` se indica el número de neuronas del bloque, 64, y con `activation` se le indica la función de activación de las neuronas, en este caso, “*relu*” (“la ReLU” pa los amigos —lo siento, no puedo evitarlo—) o dicho de otro modo, el filtro que apaga los valores negativos.

Al poner `input_dim=10`, como he comentado antes, se indica los datos que van a entrar, es decir, estamos indicado que son datos donde cada fila tiene exactamente 10 números (si lo entendí bien).

Cada una de las 64 neuronas tiene una conexión con cada uno de los 10 datos de entrada. Habrá exactamente $10 \times 64 = 640$ conexiones, o pesos (ω), operando en ese bloque y exactamente 704 parámetros totales en esta primera capa.

Capa 2

```
model.add(Dense(units=64, activation='relu'))
```

Construye la segunda capa, otra capa oculta porque no produce la salida, como si de otro `.append()` a la lista se tratase. Y de igual forma que la primera con `activation` se le indica que pase por la “la ReLU” —lo siento, me hace mucha gracia—

Capa 3

```
model.add(Dense(units=5, activation='softmax'))
```

La última capa, no es oculta, es la capa de salida (el último `.append()` a la lista). Esta vez tiene el parámetro `units` a **5** y `activation` es "softmax" con lo que ya nos está chivando que la función de activación va a coger las salidas de las 5 neuronas y los va a transformar en una distribución de porcentajes, así que cada valor puede entenderse como la probabilidad de pertenecer a cada clase, resolviendo así un problema de [clasificación multiclase](#).

Describe qué valores hemos elegido para los siguientes parámetros en la implementación [-Ir al Índice-](#)

Número de capas

Son tres capas en total como se ha definido anteriormente en el apartado [La estructura de la red](#)

Capa 1 (Capa de entrada, capa oculta), es el primer bloque Dense o capa densa que se añade al modelo.

```
model.add(Dense(units=64, activation='relu', input_dim=10))
```

Capa 2 (Capa oculta intermedia), es el segundo bloque Dense. Por apuntar, se llaman "capa oculta" porque los cálculos quedan en el interior de la red, es decir, no entrega la respuesta final como hace la última capa o capa de salida.

```
model.add(Dense(units=64, activation='relu'))
```

Capa 3 (Capa de salida), es el último bloque y la salida de la red, la encargada de dar el resultado.

```
model.add(Dense(units=5, activation='softmax'))
```

Volumen de cada capa

El **volumen de la Capa 1**, de **64 neuronas** (`units=64`), lo que significa que hay 64 funciones matemáticas preparadas para recibir y procesar los 10 datos de entrada.

El **volumen de la Capa 2**, de **64 neuronas** (`units=64`), repite el mismo tamaño que la anterior y vuelve a recibir y procesar la información que le llega de esta.

El **volumen de la Capa 3 (Capa de salida)**, de **5 neuronas** (`units=5`). Otro apunte: este volumen de la capa de salida es el más rígido porque siempre tiene que coincidir con el número de respuestas posibles que tiene el problema, ni una más, ni una menos.

Función de activación de cada capa

La **Capa 1 (entrada/oculta)** utiliza la función de activación **ReLU** (Unidad Lineal Rectificada) —*así le quitamos humor al asunto*— Se coloca a la salida de la primera capa para que la red aprenda a buscar patrones básicos al convertir todos los resultados negativos en un 0 y dejar pasar los positivos.

La **Capa 2 (oculta)** vuelve a utilizar la función de activación **ReLU**. Por el mismo motivo, al ser otra capa intermedia, debe tener la capacidad de "encender o apagar" neuronas para seguir averiguando los posibles patrones ocultos de los datos.

La **Capa 3 (salida)** utiliza la función de activación **'softmax'**. En este punto, habiendo definido la [Capa 3](#) y que responde igualmente a este apartado, ya [sabemos cuál es la respuesta a la última pregunta](#).

Optimizador

Lo primero, identificarlo en el código del enunciado, porque, hasta ahora, he definido las capas según el código, pero no he definido la línea que tiene este parámetro.

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Dentro de la función `model.compile()` vemos en el parámetro `optimizer` la función **Adam** (Adaptive Moment Estimation). El optimizador es el algoritmo encargado de **ajustar los pesos y los sesgos** de las neuronas durante el entrenamiento y para esto **Adam**, por ser de los más rápidos y robustos, es de los más utilizados por defecto en muchos frameworks de Deep Learning, como es Keras en este caso.

Función de pérdida

En la línea de código se encuentra bajo el parámetro `loss` y la función de pérdida asignada es `'categorical_crossentropy'`

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Por definición la función de pérdida es la herramienta que mide **cuánto se equivoca** el modelo, en este caso **la red**. Mide el grado de error o *castigo* del modelo al hacer las predicciones. El objetivo del entrenamiento es que este número sea lo más cercano posible a cero.

Voy a adelantarme a la última pregunta del ejercicio ([otra vez](#)), y ya que sabemos que [nuestro problema es de clasificación multiclase](#) (tenemos 5 opciones, 5 neuronas en la capa de salida) y su uso, induce a pensar que las etiquetas de los datos reales deben estar en formato de vectores `([0, 1, 0, 0, 0])` para poder compararse con los porcentajes de salidas de las 5 neuronas de la función *Softmax*.

Métrica de evaluación

Este último parámetro lo encontramos al final de `model.compile()` en `metrics=['accuracy']`.

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

La *métrica de evaluación* es el indicador que usamos para ver **cómo de bien o cómo de mal funciona la red**. La diferencia con la función de pérdida es que ésta es un número abstracto para que la máquina calcule el error y la métrica está pensada para que se entienda perfectamente, nada de abstracciones.

El valor *accuracy* es la **exactitud** y nos muestra el **porcentaje de aciertos puros** que tiene la red durante el examen, un 85% de acierto, por así decirlo, un 8.5 sobre 10 de nota —*aunque un 10 de 10 tampoco estaría mal...*—

Por último, con esta red neuronal que estamos implementando, ¿Qué tipo de problema estamos intentando resolver?

Estamos intentando resolver un problema de Clasificación Multiclase.

Como ya he explicado en la [definición de la Capa 3](#), sabemos esto gracias a que su configuración (de la [Capa 3](#)) marca 5 neuronas y función de activación **Softmax**). En resumen el hecho de tener 5 neuronas indica que se tiene que elegir entre 5 clases posibles. Y al combinarlo con **Softmax**, el modelo devolverá la probabilidad para cada una de esas 5 opciones, permitiendo identificar cuál es la más probable.

Anexo

Manuales consultados:

Máster IA y Big Data ([LinkiaFP](#))

https://campus.linkiafp.es/pluginfile.php/937019/mod_resource/content/1/IABD_MP2_T04.pdf

Keras

<https://keras.io/api/models/sequential/>

https://keras.io/api/layers/core_layers/dense/

<https://keras.io/api/metrics/>